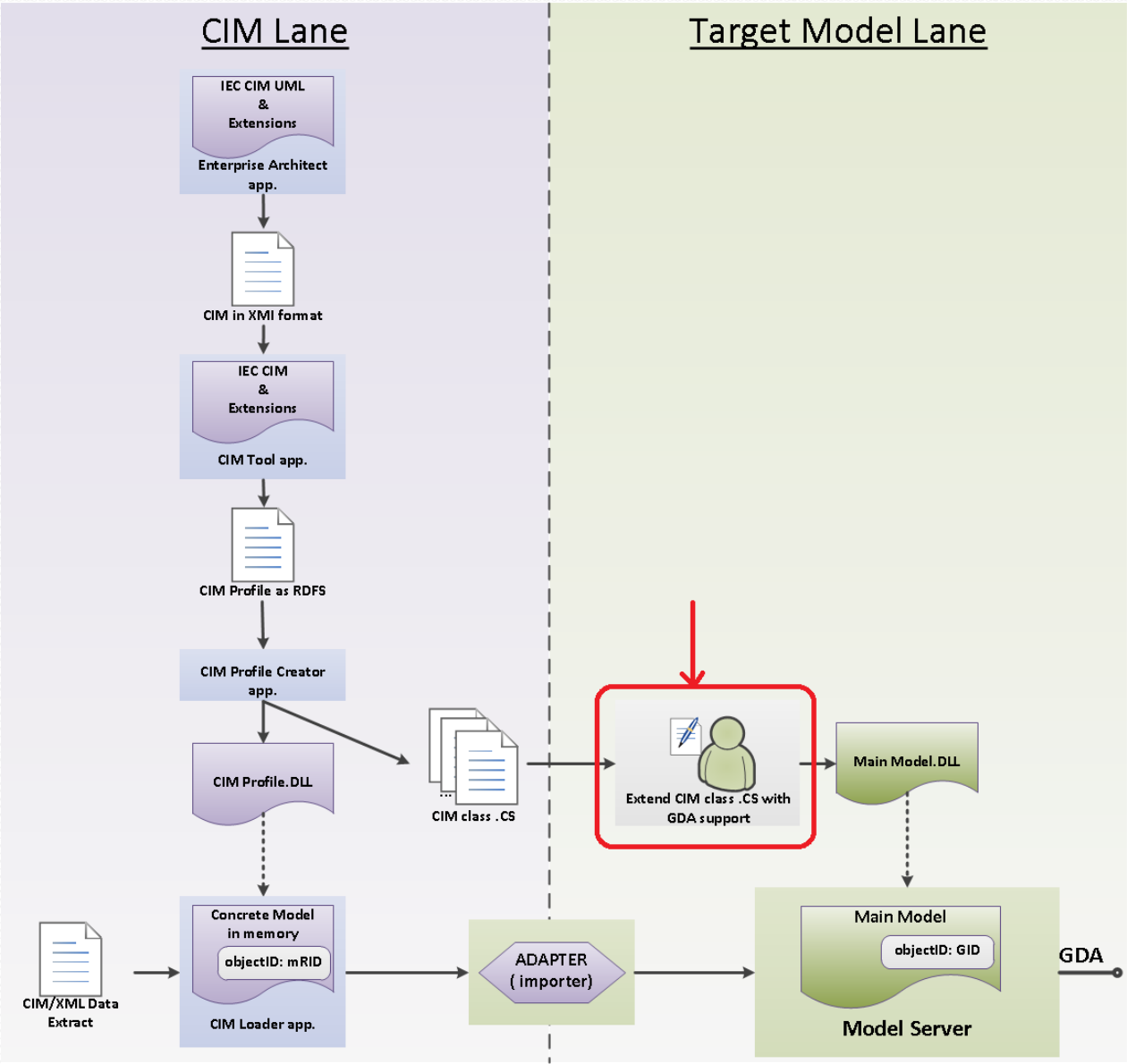
Modeli podataka u pametnim mrežama

VEŽBA 6b:

Generic Data Access podrška za čitanje podataka sa Network Model Service-a

Tok podataka pri inicijalizaciji modela elektroenergetske mreže



Metode za čitanje podataka

- Resource Query Service

- GetValues(…) – čitanje jednog resursa

```
public ResourceDescription GetValues(long resourceId,  List<ModelCode> propIds)
```

- GetExtentValues(…) – čitanje niza resursa za isti tip entiteta

```
public int GetExtentValues(ModelCode code, List<ModelCode> propIds)
```

- GetRelatedValues(…) – čitanje referenciranog resursa

```
public int GetRelatedValues(long source, List<ModelCode> propIds , Association association)
```

- GetDescendentValues(…) – čitanje niza referenciranih resursa

```
public int GetDescendentValues(List<long> sources, List<ModelCode> propIds,
                                                    List<Association> path, List<Association> tail)
```

GetValues(...)

- Čitanje jednog resursa (entiteta)

```
public ResourceDescription GetValues
      (long resourceId, List<ModelCode> propIds)
```

- `resourceId`: GID entiteta za koji se traže podaci
- `propIds`: lista traženih propertija entiteta
- Povratna vrednost: *ResourceDescription* koji sadrži GID i listu zahtevanih propertija

```
long globalId = 0x0000000300000001;

List<ModelCode> propertyIDs = new List<ModelCode>();
propertyIDs.Add(ModelCode.IDOBJ_MRID);
propertyIDs.Add(ModelCode.IDOBJ_DESCRIPTION);
propertyIDs.Add(ModelCode.EQUIPMENT_ISPRIVATE);
propertyIDs.Add(ModelCode.POWERTR_FUNC);

rd = GdaQueryProxy.GetValues(globalId, propertyIDs);
```

GetExtentValues(...)

- Čitanje niza resursa za isti tip entiteta

```
public int GetExtentValues(ModelCode code, List<ModelCode> propIds)
```

- `code:` ModelCode koji označava tip entiteta
- `propIds:` lista propertija koje se čitaju za taj tip entiteta
- Povratna vrijednost: identifikator upita (iterator) za čitanje `ResourceDescription` objekata

```
ModelCode resourceType = ModelCode.POWERTR;

List<ModelCode> propertyIDs = new List<ModelCode>();
propertyIDs.Add(ModelCode.IDOBJ_MRID);
propertyIDs.Add(ModelCode.IDOBJ_DESCRIPTION);
propertyIDs.Add(ModelCode.EQUIPMENT_ISPRIVATE);
propertyIDs.Add(ModelCode.POWERTR_FUNC);

int iteratorId = GdaQueryProxy.GetExtentValues(resourceType, propertyIDs);
```

Iteriranje kroz rezultat upita 1/2

- Metode čiji je rezultat niz *ResourceDescription*-a, ne vraćaju niz već identifkator upita. Niz može biti veoma velik i nije pogodno da klijent sve podatke prihvati od jednom. Servis čuva rezultate upita, a klijent je u mogućnosti da iterira kroz rezultujući niz.
- Da bi ovo omogućio, pored metoda koje predstavljaju upit, neophodno je da servis obezbedi metode za iteriranje kroz rezultate upita:
  - List<ResourceDescription> IteratorNext(int n, int id); - pozivom ove metode klijent zahteva od servisa da mu prosledi narednih *n ResourceDescription*-a za upit čiji je identifikator *id*
  - bool IteratorRewind(int id); - zahtev servisu da se vrati na početak rezultujućeg niza
  - int IteratorResourcesTotal(int id); - koliko ukupno postoji *ResourceDescription-a* u rezultujućem nizu
  - int IteratorResourcesLeft(int id); - koliko je ostalo nepročitanih *ResourceDescription-a*
  - bool IteratorClose(int id); - oslobađanje resursa, poziva se kada se završi rad nad upitom čiji je identifikator *id*

Iteriranje kroz rezultat upita 2/2

```
ModelCode resourceType = ModelCode.POWERTR;

List<ModelCode> propertyIDs = new List<ModelCode>();
propertyIDs.Add(ModelCode.IDOBJ_MRID);
propertyIDs.Add(ModelCode.IDOBJ_DESCRIPTION);
propertyIDs.Add(ModelCode.EQUIPMENT_ISPRIVATE);
propertyIDs.Add(ModelCode.POWERTR_FUNC);

int iteratorId = GdaQueryProxy.GetExtentValues(resourceType, propertyIDs);

int resourcesLeft = GdaQueryProxy.IteratorResourcesLeft(iteratorId);

int numberOfResources = 500;
List<ResourceDescription> result = new List<ResourceDescription>();

while (resourcesLeft > 0)
{
    List<ResourceDescription> rds = GdaQueryProxy.IteratorNext(numberOfResources, iteratorId);
    result.AddRange(rds);
    resourcesLeft = GdaQueryProxy.IteratorResourcesLeft(iteratorId);
}

GdaQueryProxy.IteratorClose(iteratorId);
```

GetRelatedValues(…) 1/3

- Čitanje niza resursa vezanih za neki resurs (entitet)

```
public int GetRelatedValues
    (long source, List<ModelCode> propIds, Association association)
```

- `source:` GID entiteta za koji tražimo povezane entitete
- `propIds:` lista propertija koje vraćamo za povezane entitete
- `association:` veza po kojoj se pronalaze povezani entiteti

- Rezultat je identifikator upita (identifikator iteratora) na osnovu koga klijent može da pročita rezultujuće *ResourceDescription-e*

GetRelatedValues(…) 2/3

- Association – objekat kojim se definišu veze između entiteta u GDA upitima
- Sadrži tri atributa: *inverse*, *propertyId*, *type*
- *bool Inverse* – nije od interesa na ovom kursu
- *ModelCode propertyId* – identifikator propertija tipa *reference* koji sadrži *source*. Vrednost tog propertija sadrži globalne identifikatore resursa koji su povezani sa *source*-om preko ovog propertija.
- *ModelCode type* – služi za filtriranje resursa koji se dobiju na osnovu *propertyId*-a. Ukoliko mu je vrednost nula ne radi se filtriranje, u suprotnom kao rezultat primene asocijacije vraćaju se samo resursi tog tipa.

```
[DataContract]
public class Association
{
    private bool inverse;
    private ModelCode propertyId;
    private ModelCode type;

    public Association()
    {
        this.inverse = false;
        this.propertyId = 0;
        this.type = 0;
    }
```

GetRelatedValues(…) 3/3

```
long source = 0x0000000200000001;

Association association = new Association();
association.PropertyId = ModelCode.LOCATION_PSRS;
association.Type = ModelCode.POWERTR;

List<ModelCode> propertyIDs = new List<ModelCode>();
propertyIDs.Add(ModelCode.IDOBJ_MRID);
propertyIDs.Add(ModelCode.IDOBJ_DESCRIPTION);
propertyIDs.Add(ModelCode.EQUIPMENT_ISPRIVATE);
propertyIDs.Add(ModelCode.POWERTR_FUNC);

int iteratorId = GdaQueryProxy.GetRelatedValues(source, propertyIDs, association);
int resourcesLeft = GdaQueryProxy.IteratorResourcesLeft(iteratorId);

int numberOfResources = 500;
List<ResourceDescription> result = new List<ResourceDescription>();

while (resourcesLeft > 0)
{
    List<ResourceDescription> rds = GdaQueryProxy.IteratorNext(numberOfResources, iteratorId);
    result.AddRange(rds);
    resourcesLeft = GdaQueryProxy.IteratorResourcesLeft(iteratorId);
}

GdaQueryProxy.IteratorClose(iteratorId);
```

GetDescendentValues(…) 1/2

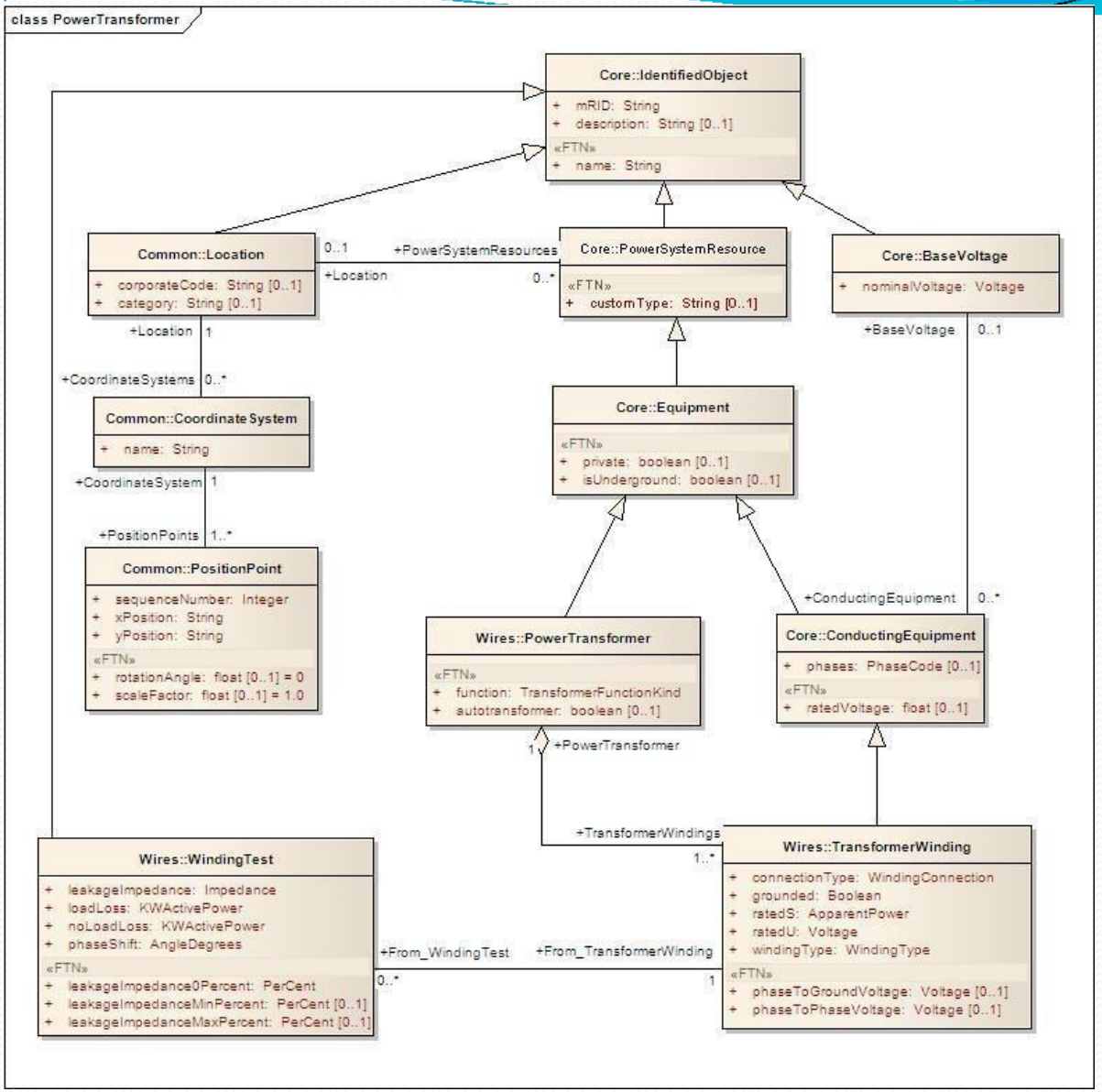
- Čitanje niza resursa vezanih za niz prosleđenih resursa (entiteta)

```
public int GetDescendentValues
        (List<long> sources, List<ModelCode> propIds,
         List<Association> path, List<Association> tail)
```

- `sources`: lista GID-ova početnih entiteta
- `propIds`: lista propertija rezultujućih entiteta
- `path`: lista asocijacija koje definišu put do povezanih entiteta
- `tail`: dodatne asocijacije za završni deo pretrage
- Povratna vrednost: identifikator upita (iterator) za čitanje ResourceDescription objekata

GetDescendentValues(…) 2/2

```
List<ModelCode> properties = new List<ModelCode>();
properties.Add(ModelCode.WINDINGTEST_LOADLOSS);
properties.Add(ModelCode.WINDINGTEST_NOLOADLOSS);
properties.Add(ModelCode.IDOBJ_MRID);

Association firstAssociation = new Association();
firstAssociation.PropertyId = ModelCode.BASEVOLTAGE_CONDEQS;
firstAssociation.Type = ModelCode.POWERTRWINDING;

Association secondAssociation = new Association();
secondAssociation.PropertyId = ModelCode.POWERTRWINDING_TESTS;
secondAssociation.Type = 0;

List<Association> path = new List<Association>() { firstAssociation, secondAssociation };
List<Association> tail = new List<Association>();

List<long> sources = new List<long>() { 0x0000000100000001, 0x0000000100000002, 0x0000000100000003 };

int iteratorId = GdaQueryProxy.GetDescendentValues(sources, properties, path, tail);
```

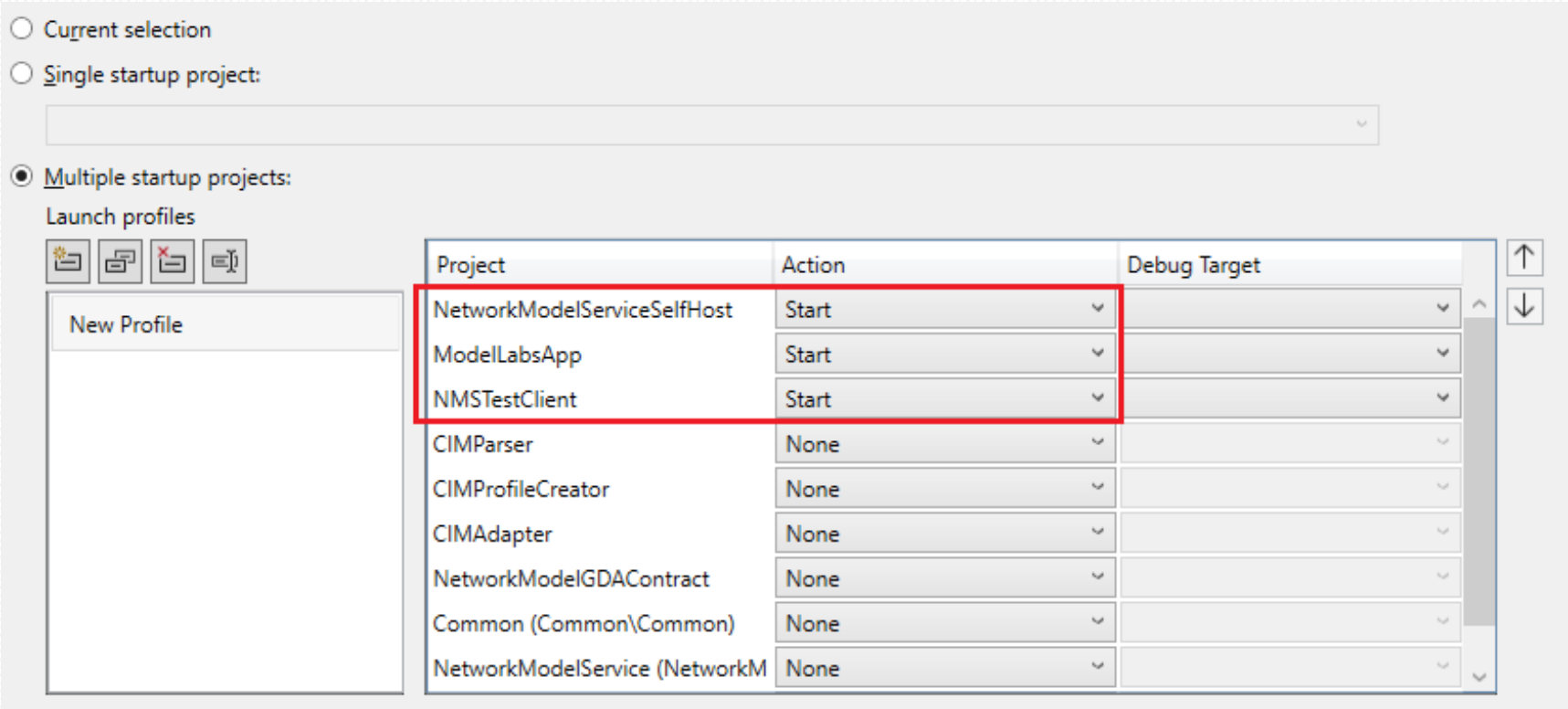

class PowerTransformer
Core::IdentifiedObject
+ mRID: String
+ description: String [0..1]
«FTN»
+ name: String
Common::Location
+ corporateCode: String [0..1]
+ category: String [0..1]
0..1
+PowerSystemResources
+Location
0..*
Core::PowerSystemResource
«FTN»
+ customType: String [0..1]
Core::BaseVoltage
+ nominalVoltage: Voltage
+Location
1
+BaseVoltage
0..1
+CoordinateSystems
0..*
Common::CoordinateSystem
+ name: String
Core::Equipment
«FTN»
+ private: boolean [0..1]
+ isUnderground: boolean [0..1]
+CoordinateSystem
1
+PositionPoints
1..*
Common::PositionPoint
+ sequenceNumber: Integer
+ xPosition: String
+ yPosition: String
«FTN»
+ rotationAngle: float [0..1] = 0
+ scaleFactor: float [0..1] = 1.0
+ConductingEquipment
0..*
Wires::PowerTransformer
«FTN»
+ function: TransformerFunctionKind
+ autotransformer: boolean [0..1]
Core::ConductingEquipment
+ phases: PhaseCode [0..1]
«FTN»
+ ratedVoltage: float [0..1]
1
+PowerTransformer
+TransformerWindings
1..*
Wires::WindingTest
+ leakageImpedance: Impedance
+ loadLoss: KWActivePower
+ noLoadLoss: KWActivePower
+ phaseShift: AngleDegrees
«FTN»
+ leakageImpedance0Percent: PerCent
+ leakageImpedanceMinPercent: PerCent [0..1]
+ leakageImpedanceMaxPercent: PerCent [0..1]
+From_WindingTest
+From_TransformerWinding
0..*
1
Wires::TransformerWinding
+ connectionType: WindingConnection
+ grounded: Boolean
+ ratedS: ApparentPower
+ ratedU: Voltage
+ windingType: WindingType
«FTN»
+ phaseToGroundVoltage: Voltage [0..1]
+ phaseToPhaseVoltage: Voltage [0..1]

Zadatak

- U okviru NMSTestClient projekta dodati funkcionalnost čitanja svih resursa čiji je tip entiteta PowerTransformer:

- Pročitati sve PowerTransformer-e sa servisa.
- Prikazati ukupan broj PoweTransformer-a.
- Za svaki od od njih prikazati:
  - GID
  - sva svojstva (*properties*) koje sadrži.

Zadatak 2

- Za PowerTransformer koji ima najmanji globalni identifikator pročitati TransformerWinding-e koji su vezani za njega. Tražiti sve propertije koje TransformerWinding sadrži.

Pokretanje projekta